

OFFICIAL

XappWindow 3.0 SDK Guide

By Poly / PIXAPoLY Software

Hey there! Welcome to the XappWindow 3.0 SDK Guide! This guide will show you how to make your very own apps for PolyOS 7 on Scratch, and how to share them with others! Before we start, this guide was 100% written by me (Poly) so apologies if it isn't totally clear or leaves some room to be desired. If you have any questions feel free to ask me on my Scratch profile or in my Discord. Thanks!

Table of Contents:

1. Getting Started
2. Setting up your window
3. Designing in Figma (Highly recommended but optional)
4. Coding the UI
5. Finishing touches + icon and XUI integration (Optional)

Chapter 1: Getting Started.

Before you begin creating your app for PolyOS 7, you must think of the general idea you are going for. What will be the main purpose of the app? How many individual UI elements will there be? Will I use the built in UI or my own custom designed UI elements? These questions can also be answered as you go, especially as you are playing around to see what you can develop. Keep some of these key things in mind however:

- This is an XappWindow API app, meaning it requires **SPRITE IMPORT** by another user of your app.
- This app is NOT PolyScript, and therefore cannot be coded directly in PolyOS 7. (Therefore by that nature it shouldn't use any high level PolyOS 7 commands!)

Starting your app:

Step 1a: Open PolyOS 7 and install a fresh installation. Make sure you have no apps open on the desktop.

Step 1b: Click see inside or remix

Step 2: Import the PolyOS 7 XappWindow 3.0 SDK Window Template Sprite using the “Import Sprite” button. Don’t do anything to it just yet!

Step 3: There should be two main sections inside the window template code. You may have to zoom out to see both. One is the core window and XUI code. Please ignore this section as you don’t have to edit any code here. Editing that code may make the window not work. You should instead be in the editing section, shown by the many documentation comments.



Where you should be: Editing Section

Don’t worry if this looks scary! You will not be changing barely any of this.

Now it’s time to edit stuff! This will make your window openable in PolyOS.

Chapter 2: Setting Up Your Window

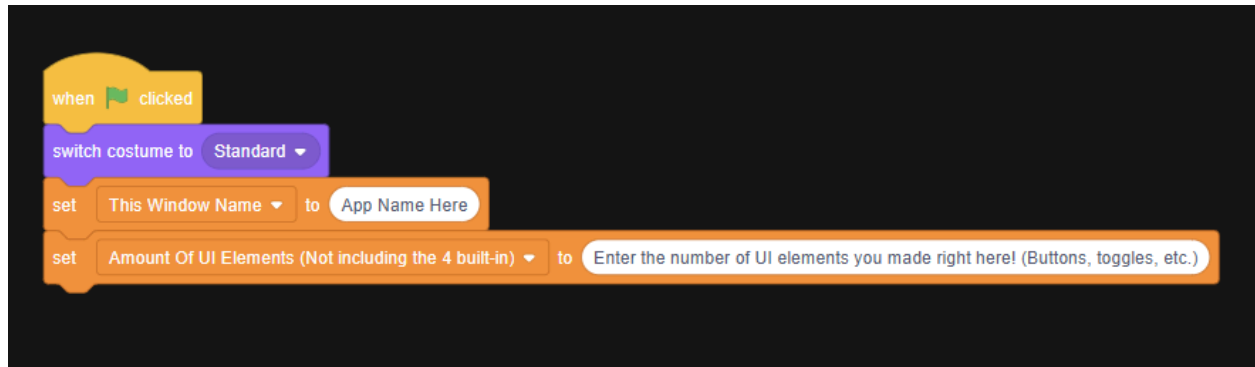
Now that you have imported the correct sprite and have the editor ready to go, it’s time to make your window openable. The very first thing is to make a global app name that you will be using in a ton of places for PolyOS to refer to your app specifically.

Step 4: Create a short, but recognizable app name for your app to be called within PolyOS. This is often just the same name as your app, (minus any special characters or unicode symbols).

Step 5: Rename the template sprite to your app name. The sprite should be called “App Name Here” as of now.

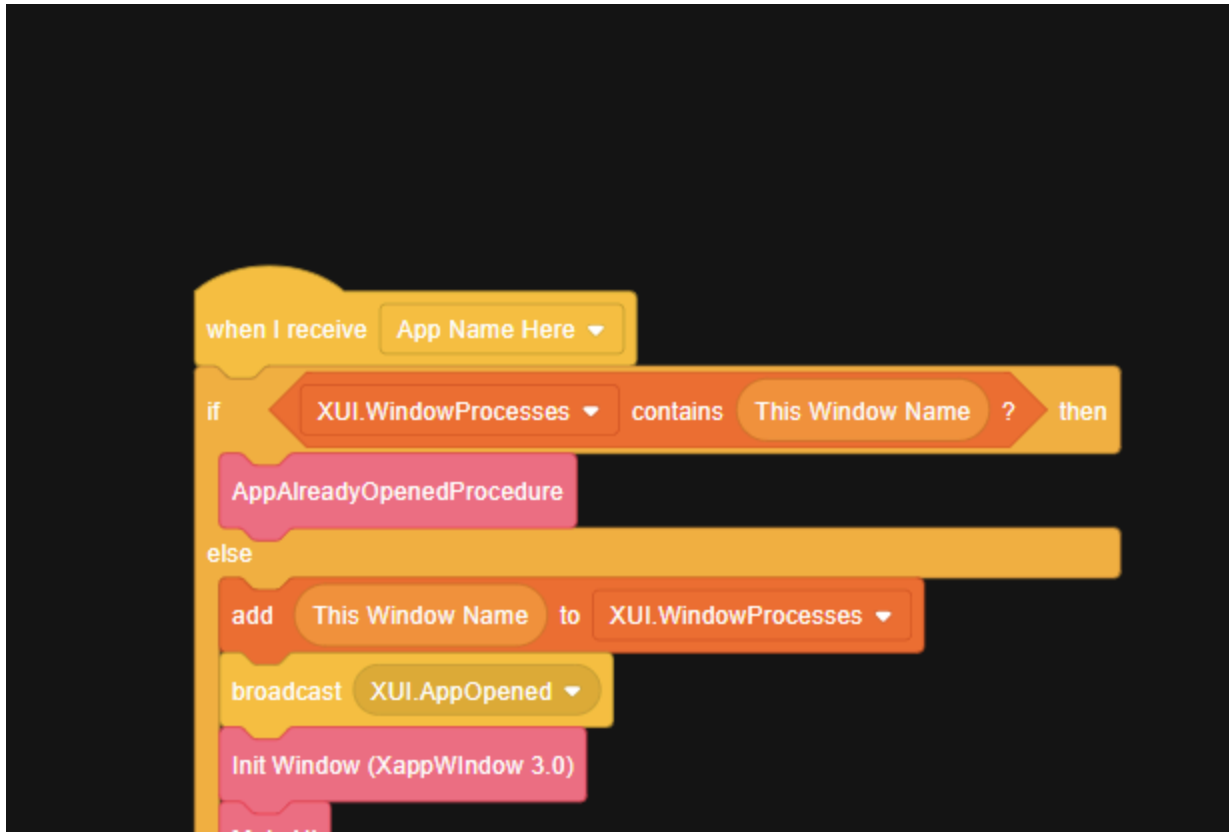
Important! Any time you see “App Name Here” Rename this EXACTLY the name of your app you decided on in step 4. This is essential for it to work.

Step 6a: Go to the cluster of variables attached to the “On flag clicked” event block. Set the “This Window Name” variable to the same exact name you decided on earlier. (See below)

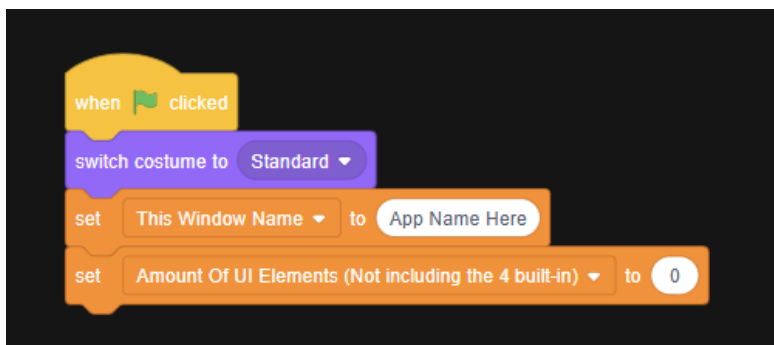


Step 6b: Unless you want to restart the project using the green flag (which you don’t lol), click the whole script in the picture above. This will set the variables to their new values.

Step 7: Go to the giant, comment heavy, main window script. It should be under a broadcast event titled “App Name Here” (see below). Make a new broadcast that has the same exact name as earlier and set it.



Step 8a: Finally, travel back to the “when flag clicked” script from earlier. Set the “Amount of UI Window Elements” variable 0 (see below). This dictates the amount of UI elements that you have in your app. This can always be changed later! In fact, there is a later step that will have you change it to the amount you have created. **For now put 0 in as it is essential to get a working demo window.**



Step 8b: Once again, don't forget to click on it to have it run and set the variable values without restarting the project!

[This step is important, as you MUST have it set to 0 before testing your blank window!]

Step 9: Now it is time to test your blank window! There are two ways to do this:

1st: Scroll up until you see the floating broadcast block. Set the broadcast block to broadcast the exact app name you set earlier. Then click on it once to open your app!

2nd. Within PolyOS 7, click on the PolyOS icon to open the home menu / side panel, then click on the "Run CMD" button. Type the exact name of the app from earlier then hit enter. Your app should open!

Your app icon will not appear in the launcher or home menu yet, we still got some designing to do! For now see if fullscreen works and how dragging your own app around feels. Congrats you made your own window in PolyOS 7!

If your app DID NOT open, check the following:

Did you correctly set every broadcast, sprite name, and the "this window name" variable to the same EXACT name?

Did you click on the set starting variables script to run it?

If your app has a bug or weird error:

Did you make sure you set the "Number of window UI elements" variable to 0 AND clicked on the script to set it?

Chapter 3: Designing in Figma.

***This step is technically optional but HIGHLY RECOMMENDED (Required for having an app nominated to be featured or put into PolyMarket) You can always just design your UI in Scratch's editor but it wouldn't match the rest of PolyOS 7 and would be pretty clunky. THIS GUIDE ONLY COVERS UI DESIGNED IN FIGMA.**

What is Figma? Figma is a UI/UX design software used to create UI for apps, games, webpages, and even PolyOS 7 itself! Figma isn't hard to use compared to other software, but still it can be hard getting started. That is why there is already a premade figma template with PolyOS buttons, sliders, switches, text, and more that you can use when making your app!

The general plan: first design your app how it would look just running in PolyOS. (Make sure you decide if you want to design both fullscreen and normal versions of the window UI) These steps will take you through the design process:

Step 10: First, head over to figma.com and make a free account or log in with Google. If you do not want to log in, it is still possible to design this way it will just be harder. It is recommended to make an account and open the template to design there.

Step 11: Once that is done, click on this link to open the pre-made PolyOS 7 UI template:

<https://www.figma.com/community/file/1588671395252825452>

Step 12a: if you **CANNOT** make an account or log in with Google, simply click on each window asset, and export them as svg (click on the side panel once the object is selected, then select “.svg” then click “export”. Do this for both windows and any buttons, switches, etc. you will need.

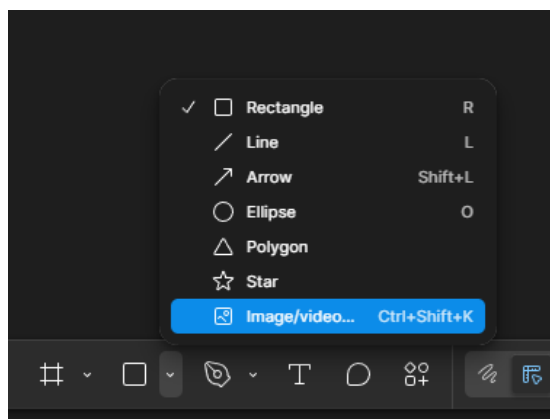
NOTE: THIS METHOD DOES NOT ALLOW YOU TO CUSTOMIZE TEXT, SO YOU ARE STUCK WITH SCRATCH'S BUILT-IN FONT. THIS EXCLUDES THE PROJECT FROM GETTING NOMINATED FOR POLYMARKET. SKIP THIS STEP IF YOU LOGGED IN.

Step 12b: Once you logged in, go back to the link from step 11. Click on “Open in Figma”. This should make a copy of the project and open it up. Feel free to follow the built-in figma tutorials (although you don't have to).

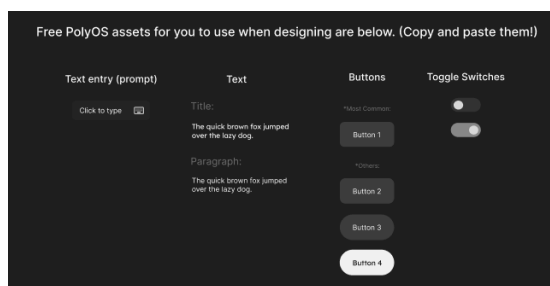
Step 13: Its design time! You can design how you generally want your app to look when running in PolyOS 7. **It is strongly recommended to use the assets provided in the project.** You can copy and paste these assets and drag them wherever you want them to be in your app. This guide will not include a full Figma tutorial. **Designing separate UI for the fullscreen is optional, and honestly more complicated. PolyOS scales the normal window UI by default! If you need another window for a separate menu or screen, just copy + paste it and move it somewhere else!**

Properties like font size, font face, color, transparency, and export settings can be changed on the side panel after an object is selected!

You can import any extra images using the place tool:



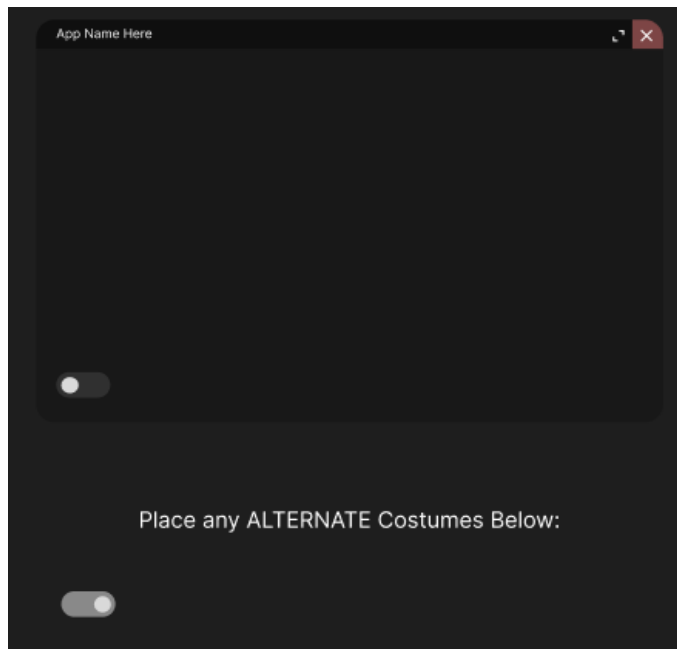
And copy and paste any of these assets to use in your app:



Step 14: Finally, after you are done designing it is time to create the alternate costumes.

[Skip this step if you do NOT have any toggle switches, or any UI elements that change its costumes]

For each UI element in your design that switches its costume (like toggle switches), put the other costumes that go with each UI element right below the window (Labeled in the section: "Alternates go here").

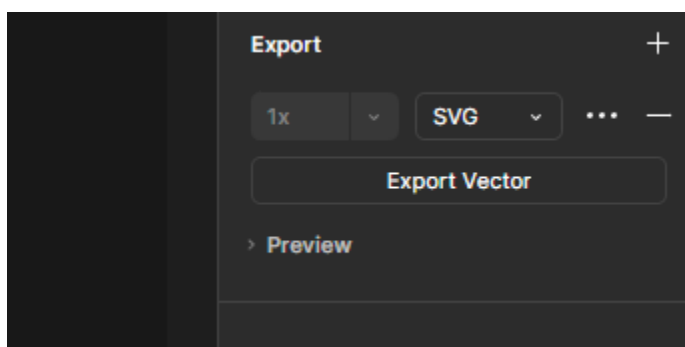


For example, if you had a toggle switch in your app that is off by default, put the on toggle switch design in that section below the window. This is so you can export both costumes into scratch for the toggle switch: one for "on" and one for "off" (see below)

Step 15a: Now it is time to export your hard work into Scratch! Drag your mouse while holding shift to select ALL of the UI elements and your window. Then right click and select "flatten".

[This will mess up any png or jpg images you imported on your app. Don't worry! In the next step we will import them separately as pngs]

Step 15b: Then go to the right side panel and scroll all the way down. Click on the drop down in the "Export as" section and select ".svg". Then click "export". This should download the svg file.



[Do this with BOTH the normal and fullscreen windows! If a .zip is downloaded, that means you need to flatten them!]

Step 16: Now head back to the running PolyOS 7 install on Scratch. Go to the costumes tab of the XappWindow 3.0 SDK Template Sprite. Import both the fullscreen svg and the normal window svg. They should both appear in the costume editor as separate costumes. **Don't worry if they are super big! This is common and can be easily fixed in the next step.**

If you have images: Import each of the images and then convert them each to vector. Next copy and paste each image into the windows you have imported. Make sure to place them where they were in Figma before you export.

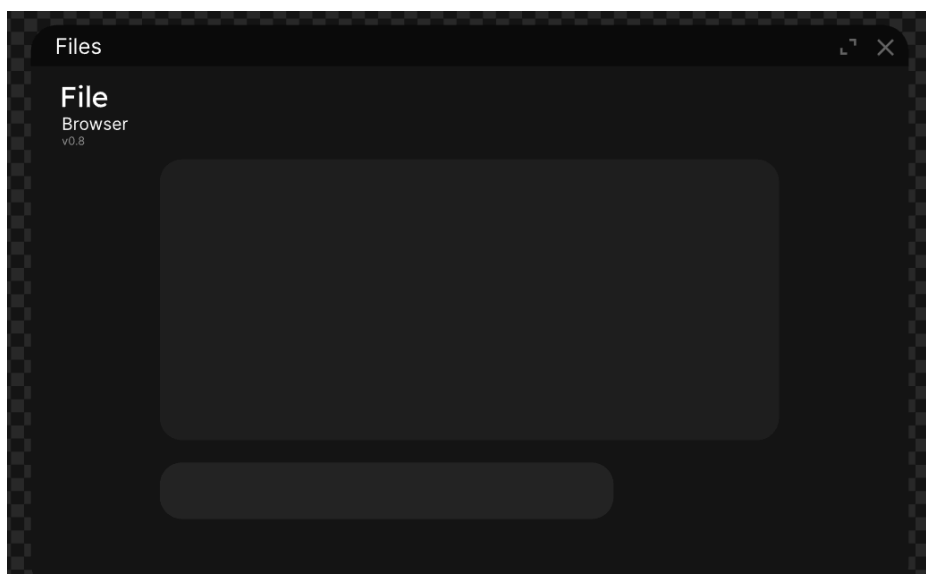
Step 17: **It is recommended to have the Scratch Addons Extension and the Costume Onion Skinning/Layer mask option enabled. This will make this step way easier. You can find Scratch addons on the chrome web store.** If you have the extension, click on the skinning button and have both the new and old window costumes right next to each other.

Then comes the important part, you need to resize the window + UI elements to match the existing window costume called "standard" or "fullscreen". Do your best to match up the sizes of the window you imported to the window that is already in the template. It's ok if it's not a 100% match in size between the two costumes. **Do this for BOTH the fullscreen and normal windows.** The fullscreen one should be easier as you only have to line it up to Scratch's built in frame guide for the project indicated by the rectangle outline in the editor).

Step 18: Next, from the costume you imported, copy only the window name from the top left corner of the window, then go to the costume titled "standard" and paste the new name you just copied and it should line up. **Do this for BOTH the fullscreen and normal windows.**

Step 19a: Now we are going to do something similar with the UI.

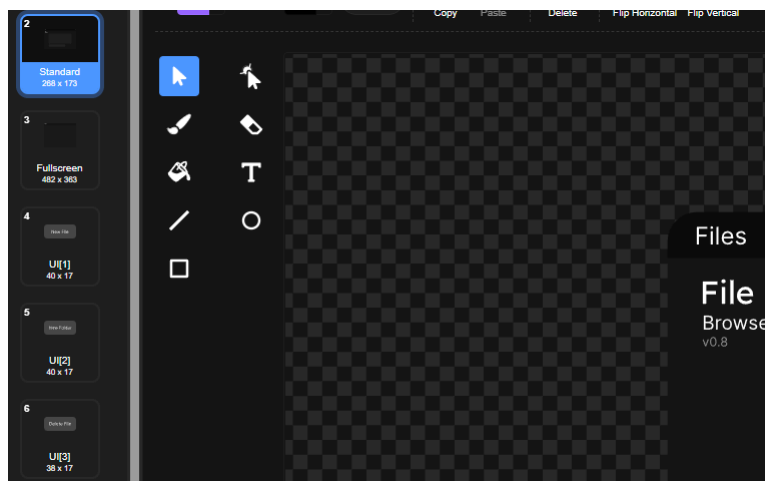
First you should know the difference between static and interactive/dynamic UI. **Static UI will NOT change nor run code.** It is usually text, images, or icons that always are in the same place EVERY time you open the app. Therefore this text can just be copied and pasted onto the costume called "standard". Go ahead and copy any static text onto the 'standard' costume (example of static UI below): **Do this for BOTH the fullscreen and normal windows.**



Step 19b: Now for the dynamic UI. These are things like buttons or switches that do something in the code and/or change. Go to the imported **normal** window. For each dynamic UI element, copy it as is, and paste it to a new costume, then delete it from the imported window.

You ONLY need to do this once for the standard window! The fullscreen window uses these same costumes. DO NOT move or center the pasted UI elements on the cross! Paste them exactly where they were when you copied them! This is so they keep with the design from earlier.

After that you should be left with an imported window with only static UI and many costumes, each being a UI element. (see an example below):



Step 20: Now we will name each UI element. This step is very important for them to work! Name each of the UI elements in this format: **UI[Number]** So the first UI Element would be called UI[1], the next one UI[2], then UI[3].. And so on...

Make sure you do this for EVERY dynamic UI element you have. For any alternate UI elements, import them now and scale them so they are exactly the same as their default counterparts. The alternates should be named with the format: UI[Number]Letter. For example, a toggle switch that is UI[2], the counterpart to the toggle switch (the opposite) would be named UI[2]A. If your UI element has more than one alternate, the keep going down the alphabet (Ex: UI[2]B, UI[2]C, etc.) After that you should have a fully designed Standard and Fullscreen window costume, and each UI element being named UI[1], UI[2], etc. with the alternates being named UI[1]A

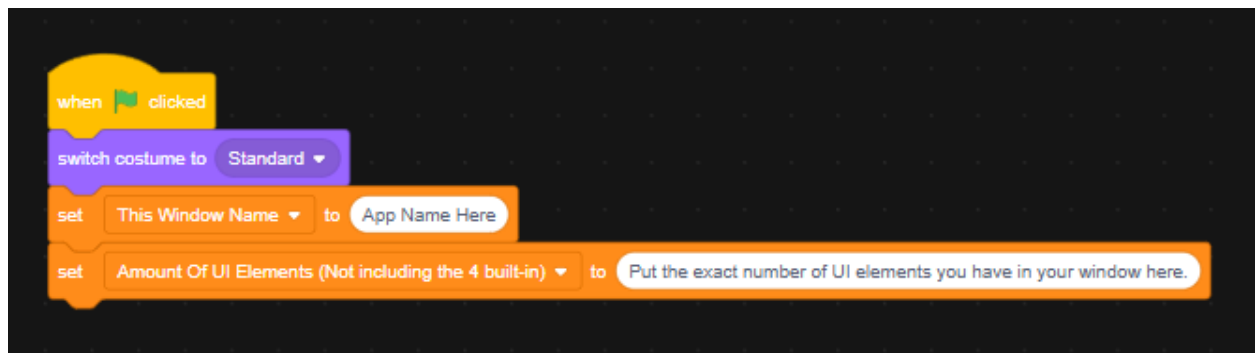
EXTRA NOTE: For UI elements where the costume changes based on a variable, and you need many alternates, it is **perfectly ok** to name the alternates in this format: UI[Number]Number . This is so the variable can easily control the costume using join blocks instead of a letter which requires if statements.

Now that you have finished designing your app fully.. Let's get to coding the UI functions!

Chapter 4: Coding the UI

It's the moment you have been waiting for! It's time to actually make your app work! Before we begin it is important to take note that this guide cannot go over every specific app's code and how to go about creating it. This guide will only go over the general practices of getting code running and getting your UI up on the screen. The rest of the app specific coding will be set up by this guide but you have to finish it. **Don't worry, for buttons, toggle switches, etc, there is already example code you can use as much as you like!** You just have to decide what those buttons do.

Step 21: Before you open your app to test what the UI will look like, we have to update the 'Number of UI Elements' variable. Scroll over in the editor to the variable section we used earlier: **Again, make sure you click on this script to run it so it sets the variables before testing!**



Yours should say 0 for the last variable. Go ahead and change it to the exact amount of UI elements you have (only dynamic) **DON'T include any alternate costumes; ONLY count one costume for each individual UI element (ex: a toggle switch would count as one, not two UI elements)***

**Ignore the "(Not including the 4 built-in)" – That is old and was back when apps had legacy UI button elements packaged with them.*

Step 22: Next, navigate to the area in the sprite shown in the picture below (scroll down a page):



Once you are here, this is where most of the coding for the UI will be done. Don't panic! The 3 chunks of code on the side are all copy and paste examples of code you can use that the guide will cover.

Step 23: This is a long and important step. Look back in the costume editor and recall what each costume is (whether it's a button, switch, etc.) Then focus on the "when I start as a clone" in the picture above. There is a chain of if-else statements already there. They each check if the clone ID is equal to a specific number. What this means is the code for each of your UI elements goes in these if-else statements. For example in the picture above: UI[1] would be the first if else statement, UI[2] would be the second one, and UI[3] would be the third. Any code under a specific if-else statement is only THAT UI element's code.

Therefore you should code what UI[1] should be doing in the first if else statement, as it checks if the clone is UI[1].

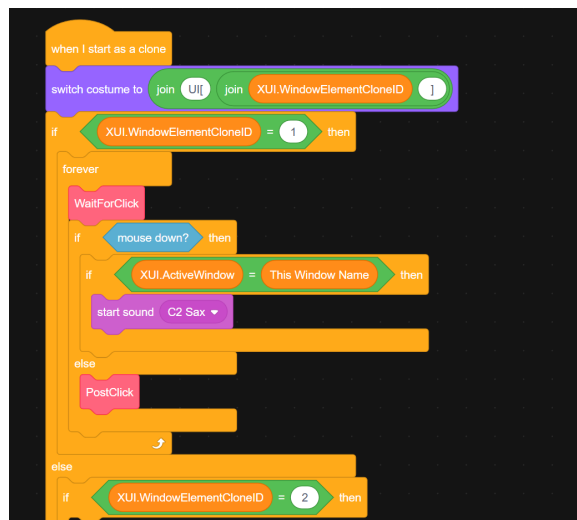
At any time if you forget what “UI[1]” or any element actually is, just look at the costume editor and see which element is named UI[1].

An example coding process is described below (you do not follow this exactly, but this will give you an idea on how to do it)

We will be coding for UI[1] therefore we will code in the first if-else statement.

1. Go back and check what UI[1] is. In this example let’s say it is a button that plays a sound.
2. Use the example located on the left side of the image for a button. Duplicate it and put it under the first if-else statement.
3. Replace the custom block that says “// WHEN CLICKED...” with the code we want to run.
4. We want it to play a sound, so we replace it with a “Play sound” block.
5. And done! Now when the button is clicked.. A sound should play!

The example code described above is shown below. Feel free to try it!



This process is relatively the same for most other UI elements. If you have more than 3, then just duplicate the if-else a few more times and type in the next numbers you need. You can have as many if-else statements you need. Just make sure to set the numbers at the top of the if-else to not have any of the same if-else statements!

For custom code, use the “// RUNS CONSTANTLY” example under that UI element, then replace the “// RUNS CONSTANTLY” block with any code you want that UI element to run while the app is open! You are also free to change the costume on any of the elements! Since it is a variable clone system, you can freely use the set costume block without the code changing!

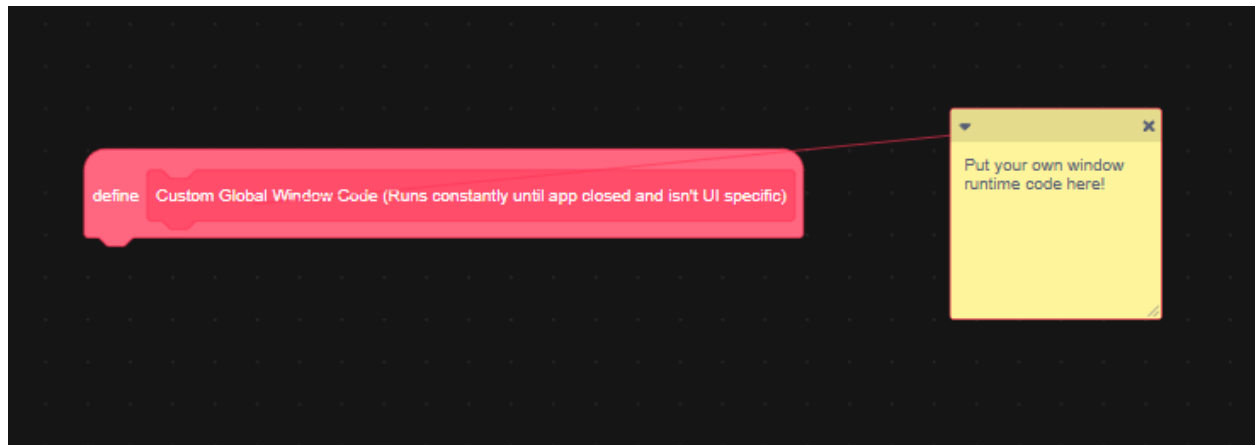
ALTERNATIVELY: For **buttons only**, if you are **100% sure** that the costume will not change and you don’t

want any sound or animation, you can use the “When this sprite clicked” example. Just put the code under the appropriate if-else statement. **This is NOT recommended but is simpler and uses less blocks if you care about optimization.**

Step 24 (OPTIONAL): If your app has background code it needs to run constantly without it being related to a specific UI element, then you can use this step to run it! Locate the custom block titled “Custom Global Window Code”. Put any code under this block to have it run constantly until the app is closed. It will not be attached or related to any UI element, only the app itself.

(an example of this would be a timer app that needs to count down in the background. You could have a button that starts it via a variable and the code for counting down would be here.)

WARNING: DO NOT PUT ANY “FOREVER” or “REPEAT UNTIL” LOOPS IN THIS CUSTOM BLOCK! THE BLOCK IS ALREADY IN A “REPEAT UNTIL APP CLOSED” LOOP; ANOTHER LOOP CAN STALL AND PREVENT THE APP FROM EVER CLOSING!



Step 25: Below are some helpful PolyOS 7 interaction coding practices that you can optionally use in your code. These can help your code be cleaner and interact with PolyOS 7.

- Use System.RAM. It's not scary! All it is is a system to prevent more variables from being used. **It only works for boolean variables (true/false)**. Simple add a flag with this format: "RAMFlagName::true" to be true and "delete (item # of (RAMFlagName::true) in System.RAM) of System.RAM" to make it false. Another part of your code can check to see if System.RAM contains "RAMFlagName::true" to see if it's true/false. This is an alternative to making a whole new variable and setting equal to true or false.
- Use broadcasts! Especially "PolyOS.GlobalUI.Sound2" or (Sound 1, 3, 4, etc.), as it plays UI sounds without having to import anything! Also it reacts to the volume slider in the home menu!
- Other broadcasts that are helpful are "PolyOS.GlobalLoadSpin" and "PolyOS.GlobalLoadSpinStop" as they can be used as an easy, no-effort loading animation!
- To have an app close itself, set XUI.CloseApp to the EXACT name of the window you set earlier.

Step 26: It's finally time to test your app. Make sure you have clicked the script that sets the number of UI elements. Double check that value is correct as well. When you are ready, click on the broadcast block you used earlier that says broadcast(exact name of app). Then your app should appear and your UI elements should show and be working!

This is the most bug-prone part of development, so don't be discouraged if it doesn't work the first try! Double check your steps and feel free to reach out and contact @PolyOS if you still can't get it working.

Common Bugs + Fixes:

Weird window costume acting as a UI Element -> Check the number of UI elements variable and click on it to set the value! Then close + reopen the app

X or Fullscreen doesn't appear -> Go back to chapter 2 and double check your steps.

UI element doesn't work or has wrong code running -> double-check that you have the correct code under the if statement for that UI element. Debug using the "say()" block.

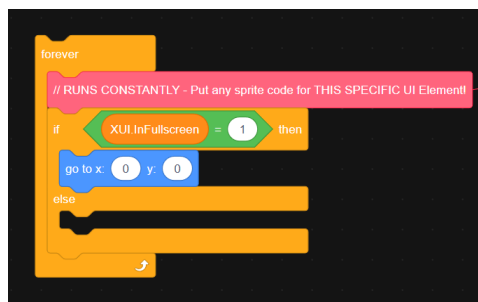
UI elements don't appear -> Check if you named the UI elements correctly! It should be "UI[1]" , "UI[2]" , etc. NO SPACES!

I don't know how exactly to code my app! -> Figure it out by testing small features at a time. If you want extra help feel free to reach out to @PolyOS!

If your app is working... Then congrats! You can officially start using the app in PolyOS 7. However, we are not done yet. We still need to optimize fullscreen, and install it into PolyOS 7 using the Xpackage installer app. Then we will next go over on how to publish your app!

Chapter 5: Finishing touches + icon and XUI integration (Optional)

Step 27 (OPTIONAL): **Fullscreen Configuration:** One optional thing some may want to do is to configure the placement of the UI elements to be different when the app is in fullscreen versus when it isn't. This can be done by simply adding the example code shown in the picture below to



each of the UI elements. Then, type the new X and Y location for each element to be when the app is in fullscreen!

By default all of the UI elements scale when in fullscreen. The code to change this can be accessed but it is on the other side where editing is discouraged! (If you can find it you can disable it but good luck).

Step 28: Finally we can install your app into PolyOS 7! Open up the app “XPackage Installer Utility” and click next. Then enter the EXACT name of your app from way earlier. The utility should install the app and it should be accessible through the community launcher app!

Step 29 (OPTIONAL): **Icon Integration:** There isn't a super clean way to do this as of now... but this process describes how to import an app icon so your app can be shown in the home menu and in the launcher. This is why most 3rd party apps will be launched through the Community Launcher. This provides an easy way to launch each app you import without there being an icon.

If you want an icon for your installation (which will not show on other people's PolyOS install unless they also follow these few steps) then do the process below:

1. Take your icon image and click on the sprite labeled “POS_UI//AppIconEngine”.
2. Go to the costumes tab and make a new costume. Call it the EXACT name of your app you decided earlier.
3. Import and copy+paste the icon image to that costume. Make sure it is the same size as all of the other costumes that have text underneath them. (ignore the ones that start with “[HOME]” for now). Then use the text from the window of your app and copy+paste it onto the icon costume and resize it so it is the same size as the text for all of the other costumes in the sprite.
4. Then make another new costume. Paste the same icon from step 3, but now resize it so it matches with all of the other smaller costumes that start with “[HOME]”. Title this new costume “[HOME]ExactNameOfYourApp” Replace “ExactNameOfYourApp”.
5. Then click on the launcher and open it. Click on “Customize App Layout” and copy and paste the current layout.
6. Add “ExactNameOfYourApp@” (replace ExactNameOfYourApp) to the start of the pasted config.
7. Hit enter and the new icon should now show!

Step 30 (OPTIONAL): **Publishing to Scratch:** If you want to share your creation in the community apps studio for PolyOS 7, this process is how to do it:

1. Go back to the XappWindow 3.0 SDK template project and **remix**
2. Click on the “Thumbnail” sprite
3. Import your own background image as a costume and copy and paste onto the “template” costume.
4. Click the “go to back layer” for the artwork so it is behind all of the text.
5. Import your own icon and replace the grey square with your icon.
6. Next, replace the title and author text in the costume editor as well
7. Then publish and share it to the official community app studio!

Thanks!

Thank you for following along to this guide written entirely by me! It took me awhile, so I hope it was helpful and you got to make your first app for PolyOS 7. Others can install it by importing the sprite into PolyOS and installing via the XPackage Install Utility. If you need any extra help, guidance, questions, etc. feel free to ask me on discord or on my Scratch profile @PolyOS.

Hope you enjoyed it! Thanks for using the XappWindow 3.0 SDK!